

Tema 4. Retardo



Contenido:

1. Introducción
2. Ciclos de Maquina (CM)
3. Ciclos de Instrucciones
4. Duración de un retardo
5. Retardo por software de un bucle
6. Diagrama de flujo de los retardos por software.
7. Estructura de un retardo
8. Retardo anidados
9. Retardos de dos bucles.
10. Retardos de tres bucles.
11. Librerías de Rutinas de retardos



Retardos

Introducción

A menudo es necesario que nuestros programas usen demoras o retardos, por ejemplo, si deseamos hacer parpadear un led cada segundo evidentemente se necesita usar un retardo de 1s.

Los retardos son prácticamente omnipresentes en nuestros programas, y hay dos formas de hacerlos:

- Por Software
- Por Hardware (TIMER)

Los retardos por Software consisten en la repetición controlada de un bucle hasta que ocurra una condición que permita salir de ese bucle. El retardo sería una rutina en la cual el microcontrolador se quede “bucleado” durante un tiempo. Es necesario usar uno o varios contadores que deberán ser decrementados, cuando dichos contadores lleguen a cero habrá concluido el retardo.

En este tema se abordará todo lo relacionado con los retardos por software en ensamblador, se aprenderá la estructura de un retardo, como calcularlos, el uso de subrutinas y técnicas que facilitaran la utilización de retardos en cualquier programa.



Retardos

Ciclos de Maquina (CM)

Los ciclos de maquina es el tiempo que al microcontrolador le toma en ejecutar una instrucción.

Formula de los Ciclo de Maquina (CM):

$$CM = 4 * \frac{1}{f_{osc}}$$

Donde:

- CM = Ciclos de Maquina
- f_{osc} = Frecuencia de oscilador (la frecuencia del cristal)

Ejemplo: ¿cual es la duración de los ciclos de maquina si el cristal es de 4Mhz?

Solución: $CM = 4 * \frac{1}{f_{osc}} \Rightarrow CM = 4 * \frac{1}{4Mhz} \Rightarrow CM = 1\mu\text{seg}$

Según el calculo anterior, al microcontrolador le tomaría 1 microsegundo en ejecutar una instrucción de un solo ciclo.



Retardos

Ciclos de instrucciones (CI)

Todas las instrucciones toman un ciclo de maquina excepto las de salto que toman dos o tres ciclos.

- Las instrucciones : `MOVLW`, `MOVWF` consumen un ciclo (1)
- Las instrucciones : `GOTO`, `BRA`, `CALL`, `RETURN` consumen dos ciclo (2), ya que se tratan de instrucciones de salto.
- La instrucción `DECFSZ` consume uno (1) ó dos (2) ó tres (3) ciclos, es decir, consume un ciclo (1) cuando no salta, consume dos ó tres (2 ó 3) ciclo cuando salta.

Vea el siguiente cuadro (tabla 4.1) y apréndaselo de memoria.

Tabla 4.1

Instrucción	Ciclos
CALL	2
MOVLW	1
MOVWF	1
DECFSZ	1 (2 ó 3)
GOTO	2
BRA	2
RETURN	2



Retardos

Duración de un retardo

La duración de un retardo por software es la cantidad de tiempo que dura el microcontrolador en ejecutar la rutina; Lo cual se logra multiplicando los ciclos de maquina (CM) por la suma de los ciclos de las instrucciones ($\sum CI$) que contenga la rutina.

Formula para calcular el retardo:

$$Retardo = CM * \sum CI$$

Donde:

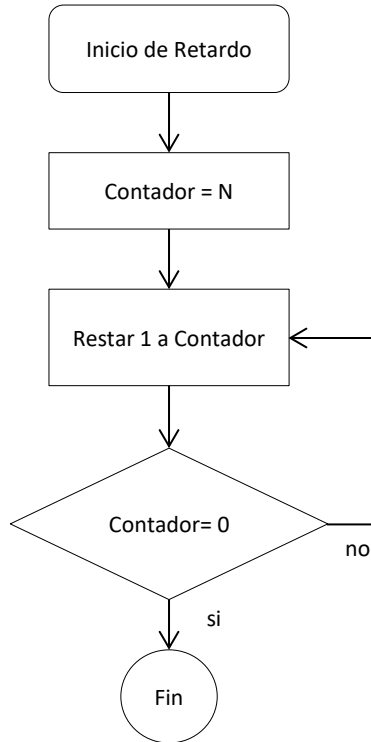
- CM = Ciclos de maquina.
- $\sum CI$ = suma de los ciclos de cada una de las instrucciones.

La suma de los ciclos de cada una de las instrucciones consiste en determinar cuantas instrucciones intervienen en la rutina y sumar los ciclos de cada una de ellas incluyendo las veces que se repitan.



Retardos

Retardo por software de un bucle



```
1  RETARDO                                ; 2*(1) del CALL
2      MOVLW      .N                        ; 1*(1)
3      MOVWF      CONTADOR                  ; 1*(1)
4  BUCLE  DECFSZ   CONTADOR, F              ; 1*(N-1) + 2*(1)
5      BRA        BUCLE                    ; 2*(N-1)
6      RETURN                               ; 2*(1)
```

Comentario:

Observe en el diagrama anterior como a una variable Contador se le asigna un numero N, posteriormente esta variable se decrementa y se pregunta si ha llegado a 0, si no ha llegado a 0 entonces se vuelve a decrementar, y así sucesivamente hasta que llegue a 0 en cuyo caso es el FIN del retardo. El programa se quedo en un bucle dando vueltas N veces, hasta que la variable contador llegue a cero.



Retardos

Ejemplo

Calcule la duración del siguiente retardo, con un cristal de 20Mhz

```

1  RETARDO
2  MOVLW    .200
3  MOVWF    CONTADOR
4  BUCLE
5  DECFSZ   CONTADOR, F
6  BRA      BUCLE
7  RETURN
  
```

Solución:

```

1  RETARDO                ; 2*(1)=2 del CALL
2  MOVLW    .200          ; 1*(1)=1
3  MOVWF    CONTADOR      ; 1*(1)=1
4  BUCLE
5  DECFSZ   CONTADOR, F   ; [1*(199)]+[2*(1)] =199+2=201
6  BRA      BUCLE         ; 2*(199)=398
7  RETURN                ; 2*(1)=2
  
```

$$Retardo = 4 * \frac{1}{20Mhz} * 605$$

$$Retardo = 121\mu Seg$$



Retardos

Retardo anidados

Estos retardos son parecidos a los de un bucle con la salvedad de que tienen ciclos anidados (un bucle dentro de otro bucle). A continuación, se presentan los retardos anidados, es decir con mas de un bucle.

Retardo de dos Bucles:

```

1. RETARDO                ; 2*(1)=2 del CALL
2.   MOVLW    .M           ; 1*(1)=1
3.   MOVWF    CONTADOR_2   ; 1*(1)=1
4. BUCLE_2
5.   MOVLW    .N           ; 1*(1)=1
6.   MOVWF    CONTADOR_1   ; 1*(1)=1
7. BUCLE_1
8.   DECFSZ   CONTADOR_1,F ; [1*(N-1)]+[2*(1)] =N-1+2 =N+1
9.   BRA      BUCLE_1     ; 2*(N-1)= 2N-2
10.  DECFSZ   CONTADOR_2,F ; [1*(M-1)]+[2*(1)] =M-1+2 =M+1
11.  BRA      BUCLE_2     ; 2*(M-1)= 2M-2
12.  RETURN                ; 2*(1)=2
  
```

$$\sum CI = 3NM + 4M + 5$$

Retardo de tres Bucles:

```

1. RETARDO                ; 2*(1)=2 del CALL
2.   MOVLW    .M           ; 1*(1)=1
3.   MOVWF    CONTADOR_3   ; 1*(1)=1
4. BUCLE_3
5.   MOVLW    .M           ; 1*(1)=1
6.   MOVWF    CONTADOR_2   ; 1*(1)=1
7. BUCLE_2
8.   MOVLW    .N           ; 1*(1)=1
9.   MOVWF    CONTADOR_1   ; 1*(1)=1
10. BUCLE_1
11.  DECFSZ   CONTADOR_1,F ; [1*(N-1)]+[2*(1)] =N-1+2 =N+1
12.  BRA      BUCLE_1     ; 2*(N-1)= 2N-2
13.  DECFSZ   CONTADOR_2,F ; [1*(M-1)]+[2*(1)] =M-1+2 =M+1
14.  BRA      BUCLE_2     ; 2*(M-1)= 2M-2
15.  DECFSZ   CONTADOR_3,F ; [1*(P-1)]+[2*(1)] =P-1+2 =P+1
16.  BRA      BUCLE_3     ; 2*(P-1)= 2P-2
17.  RETURN                ; 2*(1)=2
  
```

$$\sum CI = 3NMP + 4MP + 4P + 5$$



Retardos

Ejercicio

Realice un cuadro comparativo con la duración máxima de retardos de 1,2 y 3 bucles con un cristal de 20Mhz

Solución: Se utiliza el valor máximo para los tres contadores

- N=255
- M=255
- P=255

Bucle	$\sum_{CI}$	Retardo
1	770	0,154ms
2	196100	39,22ms
3	50005250	10seg



Retardos

Ejercicio

Elabore una sub rutina para generar una señal cuadrada de 10Hz.

Solución:

$$\frac{1}{10Hz} = 100ms$$

Se requiere un retardo de 50ms

```
SEÑAL_10HZ
    BSF    SALIDA
    CALL   RETARDO_50MS
    BCF    SALIDA
    CALL   RETARDO_50MS
    BRA    SEÑAL_10HZ
```

```
RETARDO_50MS
    MOVLW  .2
    MOVWF  CONTADOR_3
BUCLE_3_RETARDO_50MS
    MOVLW  .252
    MOVWF  CONTADOR_2
BUCLE_2_RETARDO_50MS
    MOVLW  .164
    MOVWF  CONTADOR_1
BUCLE_1_RETARDO_50MS
    DECFSZ CONTADOR_1,F
    BRA    BUCLE_1_RETARDO_50MS
    DECFSZ CONTADOR_2,F
    BRA    BUCLE_2_RETARDO_50MS
    DECFSZ CONTADOR_3,F
    BRA    BUCLE_3_RETARDO_50MS
    NOP
    NOP
    NOP
    RETURN
```



Retardos

Ejercicio

Elabore una sub rutina para generar una señal cuadrada de 10Hz. Utilice un pulsador para iniciar y finalizar la rutina.

Solución:

$$\frac{1}{10\text{Hz}} = 100\text{ms}$$

Se requiere un retardo de 50ms

```

BCF    BANDERA,0
PULSADOR
BTFSC  P1
BRA    PULSADOR
CALL   RETARDO_ANTI_REBOTE
SOLTAR_PULSADOR
BTFSS  P1
BRA    SOLTAR_PULSADOR
CALL   SEÑAL_10HZ
BRA    PULSADOR
  
```

```

SEÑAL_10HZ
BSF    SALIDA
CALL   RETARDO_50MS
BCF    SALIDA
CALL   RETARDO_50MS

BTFSS  BANDERA,0
BRA    SEÑAL_10HZ

BCF    SALIDA
CALL   RETARDO_ANTI_REBOTE
SALIR_SOLTAR_PULSADOR
BTFSS  P1
BRA    SALIR_SOLTAR_PULSADOR
BCF    BANDERA,0
RETURN
  
```

```

RETARDO_50MS
MOVLW  .2
MOVWF  CONTADOR_3
BUCLE_3_RETARDO_50MS
MOVLW  .252
MOVWF  CONTADOR_2
BUCLE_2_RETARDO_50MS
MOVLW  .164
MOVWF  CONTADOR_1
BUCLE_1_RETARDO_50MS
BTFSS  P1
BRA    SALIR_RETARDO_50MS
DECFSZ CONTADOR_1,F
BRA    BUCLE_1_RETARDO_50MS
DECFSZ CONTADOR_2,F
BRA    BUCLE_2_RETARDO_50MS
DECFSZ CONTADOR_3,F
BRA    BUCLE_3_RETARDO_50MS
NOP
;NOP
;NOP
RETURN
  
```

```

SALIR_RETARDO_50MS
BSF    BANDERA,0
RETURN
  
```

Tema 5. Comparación de Registros



Contenido:

- Introducción
- Igual a cero
- Igual a otro registro
- Menor que otro registro
- Mayor que otro registro
- Resumen
- Ejercicios

$A > B$
 $A \geq C$
 $A = .32$
 $< A \quad B \leq A$

Comparación de registros

Introducción

En la mayoría de la lógica de programación, se requiere implementar la comparación de un dato con otro, alojados en registros de memoria, por lo tanto, Saber comparar un registro con otro es de vital importancia a la hora de elaborar un programa.

Las comparaciones permiten determinar relaciones entre datos: si ambos datos son iguales o si un dato es mayor que otro, inclusive si es mayor o igual que otro y, por supuesto, también se podría determinar si es menor o menor igual que otro registro o dato.

El programador (el que crea el código) decide cual comparación hacer según su lógica, una vez determinado el resultado de la comparación, se procederá a realizar las acciones deseadas. Existe una infinidad de ejemplo de comparaciones en un programa en ensamblador. Para hacerse una idea se podrían enunciar los siguientes ejemplos:

Ejemplo 1: se desea saber si la temperatura ha llegado a un cierto valor, en ese caso se tendrá que comparar si la temperatura coincide con el valor deseado. Para ello habrá que comparar un dato (temperatura) con otro dato (valor deseado). Nota: supongamos que la medición de la temperatura previamente se obtuvo de una conversión analógica-digital y que el microcontrolador proceso la conversión y calculo la temperatura guardando el resultado en un registro.

Ejemplo 2: Durante una comunicación serial se desea saber si un carácter coincide con otro carácter. Para ello se tendrá que comparar ambos datos para determinar si son iguales.

Ejemplo 3: En un menú de opciones, se quiere saber cual tecla fue presionada, para ello se comparará una lista de números con el dato ingresado por un teclado.

Ejemplo 4: cuando se requiere hacer un control de una variable, la cual debe permanecer en un determinado rango. Se tendría que comparar si esta por debajo o por encima del rango permitido.

A>B
A≥C
A=,32
<A B≤A

Comparación de registros

Registro STATUS

5.3.6 STATUS REGISTER

The STATUS register, shown in Register 5-2, contains the arithmetic status of the ALU. As with any other SFR, it can be the operand for any instruction.

If the STATUS register is the destination for an instruction that affects the Z, DC, C, OV or N bits, the results of the instruction are not written; instead, the STATUS register is updated according to the instruction performed. Therefore, the result of an instruction with the STATUS register as its destination may be different than intended. As an example, `CLRF STATUS` will set the Z bit and leave the remaining Status bits unchanged ('000u u1uu').

It is recommended that only `BCF`, `BSF`, `SWAPF`, `MOVFF` and `MOVWF` instructions are used to alter the STATUS register because these instructions do not affect the Z, C, DC, OV or N bits in the STATUS register.

For other instructions that do not affect Status bits, see the instruction set summaries in Table 26-2 and Table 26-3.

Note: The C and DC bits operate as the Borrow and Digit Borrow bits, respectively, in subtraction.

Fuente: DataSheet. pagina 73

$A > B$
 $A \geq C$
 $A = .32$
 $C < A$ $B \leq A$

Comparación de registros

Registro STATUS

REGISTER 5-2: STATUS REGISTER

U-0	U-0	U-0	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x
—	—	—	N	OV	Z	DC ⁽¹⁾	C ⁽²⁾
bit 7							
							bit 0

bit 7-5 **Unimplemented:** Read as '0'

bit 4 **N:** Negative bit
 This bit is used for signed arithmetic (2's complement). It indicates whether the result was negative (ALU MSB = 1).
 1 = Result was negative
 0 = Result was positive

bit 3 **OV:** Overflow bit
 This bit is used for signed arithmetic (2's complement). It indicates an overflow of the 7-bit magnitude which causes the sign bit (bit 7 of the result) to change state.
 1 = Overflow occurred for signed arithmetic (in this arithmetic operation)
 0 = No overflow occurred

bit 2 **Z:** Zero bit
 1 = The result of an arithmetic or logic operation is zero
 0 = The result of an arithmetic or logic operation is not zero

bit 1 **DC:** Digit Carry/Borrow bit⁽¹⁾
 For ADDWF, ADDLW, SUBLW and SUBWF instructions:
 1 = A carry-out from the 4th low-order bit of the result occurred
 0 = No carry-out from the 4th low-order bit of the result

bit 0 **C:** Carry/Borrow bit⁽²⁾
 For ADDWF, ADDLW, SUBLW and SUBWF instructions:
 1 = A carry-out from the Most Significant bit of the result occurred
 0 = No carry-out from the Most Significant bit of the result occurred

Note 1: For Borrow, the polarity is reversed. A subtraction is executed by adding the 2's complement of the second operand. For rotate (RRF, RLF) instructions, this bit is loaded with either bit 4 or bit 3 of the source register.

2: For Borrow, the polarity is reversed. A subtraction is executed by adding the 2's complement of the second operand. For rotate (RRF, RLF) instructions, this bit is loaded with either the high or low-order bit of the source register.

$A > B$
 $A \geq C$
 $A = .32$
 $A < B$

Comparación de registros

Registro STATUS

REGISTER 5-2: STATUS REGISTER

U-0	U-0	U-0	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x
—	—	—	N	OV	Z	DC ⁽¹⁾	C ⁽²⁾
bit 7						bit 0	

La bandera Z: se pone automáticamente a uno si el resultado es cero.

La bandera C vendría siendo el 9no bit:

En la Resta actúa como bit de préstamo y se pone:

- Automáticamente a uno cuando no hay préstamo (el resultado es positivo)
- Automáticamente a cero cuando hay préstamo (resultado negativo)

En la Suma actúa como bit de llevada y se pone:

- Automáticamente a uno cuando hay llevada.
- Automáticamente a cero cuando no hay llevada.

$A > B$
 $A \geq C$
 $A =_{32}$
 $A < B$

Comparación de registros

Cuando se esta comparando, se pueden tener los siguientes casos , ya sea:

- Comparar un registro con otro registro.
- Comparar un dato con un registro.
- Comparar un dato con otro dato.

En cualquiera de los casos el procedimiento es similar. Se tiene que realizar una operación matemática y comprobar el estado de las banderas C y Z. la operación ideal es la resta con la instrucción SUBWF

De manera general cuando se realiza una operación (con las instrucciones) las banderas C, DC y Z del registro STATUS son afectadas. El estado lógico de estas banderas es puesto a uno o a cero automáticamente por la CPU del microcontrolador dependiendo del resultado de la operación, cada bandera tiene su propio comportamiento.

Para saber cual bandera es afectada, en la tabla de instrucciones se especifican las banderas (flag) que son afectadas

$A > B$
 $A \geq C$
 $A = .32$
 $A < B$

Comparación de registros

Igual a cero

Pasos para comparar si un registro es igual a cero o si es distinto de cero:

- Mover el registro sobre si mismo:** se realiza con la instrucción MOVF donde se coloca el registro que se esta comparando y se selecciona como destino el propio registro (d=1 ó d=f) es decir se mueve el dato sobre si mismo, esto hará que la bandera Z sea modificada por la instrucción MOVF
- testear la bandera Z:** si el registro es igual a cero Z se pone automáticamente a 1, caso contrario Z se pondrá a cero; Luego se verifica si Z es igual a uno con la instrucción BTFSS; También se puede usar BTFSC para ver si Z es cero. La elección de una u otra instrucción dependerá de la lógica que tenga pensado realizar el programador ya sea para comprobar si es igual o distinto de cero.

```

MOVF    A, F
BTFSS   STATUS, Z
GOTO    NO_ES_IGUAL_A_CERO
GOTO    IGUAL_A_CERO
  
```

Nota:
 A=0 Entonces Z=1 y **salta**
 A≠0 Entonces Z=0 y no salta

```

MOVF    A, F
BTFSC   STATUS, Z
GOTO    IGUAL_A_CERO
GOTO    NO_ES_IGUAL_A_CERO
  
```

Nota:
 A=0 Entonces Z=1 y **no salta**
 A≠0 Entonces Z=0 y **salta**

9.4.1 Comprobar que un registro vale 0

Para saber si un registro vale 0 hay que cargar el registro sobre sí mismo mediante la instrucción “movf” (que es la única que posiciona flags), con lo cual se logra posicionar el flag Z sin variar su contenido. Ejemplo:

```

movf    Registro,F      ; (Registro) → (Registro) y posiciona flag Z.
btfss   STATUS,Z        ; ¿Es cero?, ¿Z=1?
goto     NoEsCero        ; No.
EsCero   ; Si es cero y ejecuta la parte del programa correspondiente.
.....
NoEsCero ; No es cero y ejecuta esta parte del programa.
.....
  
```

Fuente: Libro Microcontrolador PIC16F84 - Desarrollo de proyectos
ELECTROX.COM.AR.pdf pagina 122

$A > B$
 $A \geq C$
 $A = .32$
 $< A$ $B \leq A$

Comparación de registros

Igual a otro registro

Pasos para comprobar si un registro es igual a otro registro

- Restar el dato de ambos registros:** implica realizar la operación matemática resta con la instrucción SUBWF, pero previamente hay que mover el otro registro a W con la instrucción MOVF
- testear la bandera Z:** se verifica si Z es igual a uno con la instrucción BTFSS; También se puede usar la instrucción BTFSC para ver si Z es cero. La elección de una u otra instrucción dependerá de la lógica que tenga pensado realizar el programador ya sea para comprobar si es igual o distinto

```

MOVF  B, W      ; B=W
SUBWF  A, W      ; A-W=A-B
BTFSS  STATUS, Z
GOTO   A_NO_ES_IGUAL_B
GOTO   A_ES_IGUAL_B
  
```

Nota:

Si $A=B$ Entonces $A-B=0$ luego $Z=1$ y **salta**
 Si $A \neq B$ Entonces $A-B \neq 0$ luego $Z=0$ y no salta

```

MOVF  B, W      ; B=W
SUBWF  A, W      ; A-W=A-B
BTFSC  STATUS, Z
GOTO   A_ES_IGUAL_B
GOTO   A_NO_ES_IGUAL_B
  
```

Nota:

Si $A=B$ Entonces $A-B=0$ luego $Z=1$ y **no salta**
 Si $A \neq B$ Entonces $A-B \neq 0$ luego $Z=0$ y **salta**

9.4.2 Comprobar igualdad entre dos registros

Por ejemplo para comprobar si el contenido de los registros Registro1 y Registro2 son iguales el programa a realizar sería:

```

movf   Registro1, W      ; W se carga con el valor de Registro1.
subwf  Registro2, W      ; (Registro2) - (Registro1) → (W) y además posiciona flag Z.
btfsc  STATUS, Z        ; Si Z = 0, salta ya que no son iguales.
goto   Iguales           ; Si, son iguales y salta a ejecutar lo correspondiente.
  
```

NoIguales

Iguales

En lugar de la instrucción `subwf Registro2, W` se podría haber utilizado la instrucción `xorwf Registro2, W` con idéntico resultado.

Fuente: Libro Microcontrolador PIC16F84 - Desarrollo de proyectos
ELECTROX.COM.AR.pdf pagina 122-123

$A > B$
 $A \geq C$
 $A = .32$
 $A < B$

Comparación de registros

Menor que otro registro

Pasos para la comparación de un registro si es menor que otro registro

- Restar el dato de ambos registros:** implica realizar la operación matemática resta con la instrucción SUBWF, pero previamente hay que mover el dato del otro registro a W con la instrucción MOVWF
- Testear la bandera C:** Se verifica el valor de la bandera C con la instrucción BTFSS o BTFSC

```

MOVWF    B, W    ; B=W
SUBWF    A, W    ; A-W=A-B
BTFSC    STATUS, C ; C=0
GOTO     A_NO_ES_MENOR_B
GOTO     A_ES_MENOR_B
  
```

Nota:

Sea A-B si A<B Entonces C=0 y **Salta**

```

MOVWF    B, W    ; B=W
SUBWF    A, W    ; A-W=A-B
BTFSS    STATUS, C ; C=0
GOTO     A_ES_MENOR_B
GOTO     A_NO_ES_MENOR_B
  
```

Nota:

Sea A-B si A<B Entonces C=0 y **no salta**

A>B
A≥C
A=,32
<A B≤A

Comparación de registros

Mayor que otro registro

Pasos para la comparación de un registro si es mayor que otro registro

1. **Restar el dato de ambos registros:** implica realizar la operación matemática resta con la instrucción SUBWF, pero previamente hay que mover el dato del otro registro a W con la instrucción MOVF
2. **Testear la bandera Z y C:** Se verifica el valor de las banderas C y de Z con la instrucción BTFSS o BTFSC

```
MOVF    B,W    ;B=W
SUBWF   A,W    ;A-W=A-B
BTFSS   STATUS,C ;C=1?
GOTO    A_NO_ES_MAYOR_B

BTFSS   STATUS,Z ;Z=0?
GOTO    A_ES_MAYOR_B
GOTO    A_NO_ES_MAYOR_B
```

Nota:

Sea A-B si A>B Entonces C=1 y Z=0

$A > B$
 $A \geq C$
 $A = .32$
 $< A \quad B \leq A$

Comparación de registros

Mayor que o Menor que un registro

9.4.3 Comprobar que un registro es mayor o menor que otro

En este caso hay que realizar la resta de ambos. Así siendo A y B dos registros cualesquiera y haciendo la operación (A-B) se tienen las siguientes posibilidades.:

Operación (A - B)	Resultado	Bits de Carry y Zero
A > B	Positivo	C = 1 y Z = 0
A < B	Negativo	C = 0 y Z = 0
A = B	Cero	C = 1 y Z = 1

Tabla 9-1 Resultado de la comparación de dos registros

El siguiente fragmento de programa proporciona un salto a la etiqueta "MayorIgual" cuando el (RegistroA) $\geq$ (RegistroB) y salta a "Menor" cuando el (RegistroA) < (RegistroB):

```

movf   RegistroB,W      ; (RegistroB) → (W).
subwf  RegistroA,W      ; (RegistroA) - (RegistroB) → (W).
btfss  STATUS,C         ; ¿C=1?, ¿resultado positivo?, ¿(RegistroA) ≥ (RegistroB)?
goto   Menor
MayorIgual                ; C=1. Entonces ha sido (RegistroA) ≥ (RegistroB).
.....
Menor                    ; C=0. Entonces ha sido (RegistroA) < (RegistroB).
.....

```

Fuente: Libro Microcontrolador PIC16F84 - Desarrollo de proyectos ELECTROX.COM.AR.pdf pagina 123

$A > B$
 $A \geq C$
 $A = .32$
 $< A \quad B \leq A$

Comparación de registros

Resumen

Se presenta una tabla con el resultado de las banderas C y Z cuando se ejecuta la instrucción SUBWF, para cada uno de los casos en que A y B son dos registros y al efectuar A-B se tienen los siguientes casos:

Operación A-B	Bandera C	Bandera Z
A=B	C=1	Z=1
A>B	C=1	Z=0
A<B	C=0	Z=0
A≥B	C=1	Z=0 ó Z=1
A≤B	C=0	Z=1

***Nota:** Para cada caso basta con comprobar la bandera que esta en rojo.

****Nota:** importante A-B ≠ B-A

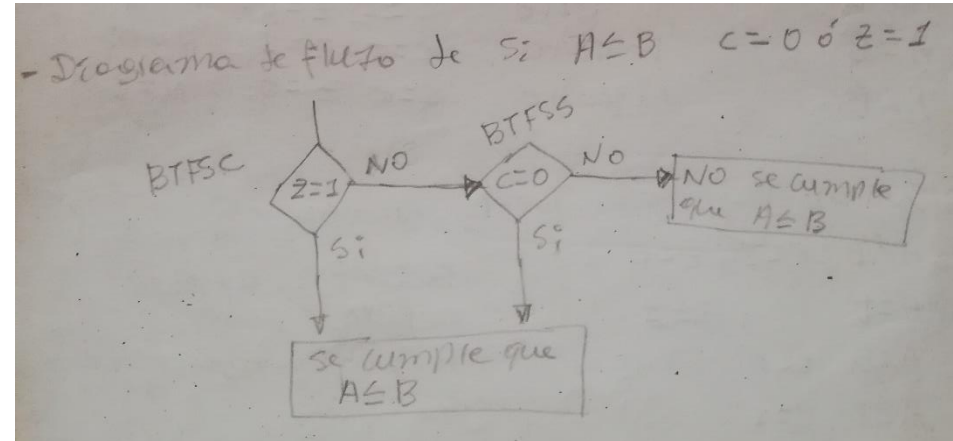
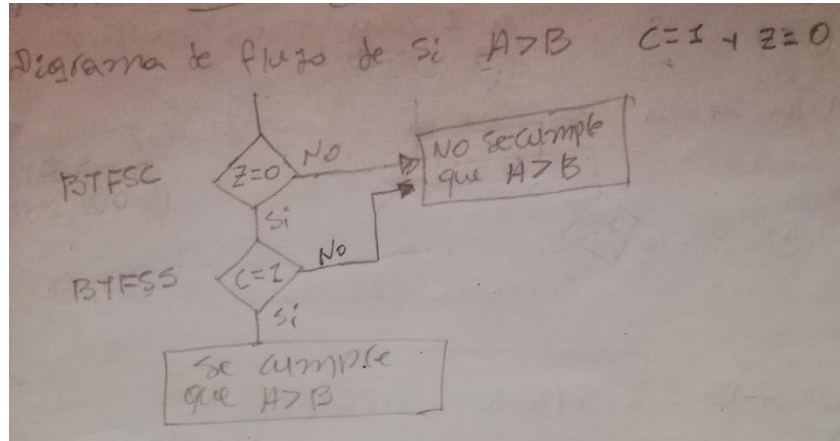
; A-B
 MOVF B, W ; B=W
 SUBWF A, W ; A-W=A-B

; B-A
 MOVF A, W ; A=W
 SUBWF B, W ; B-W=B-A

$A > B$
 $A \geq C$
 $A = .32$
 $C < A$ $B \leq A$

Comparación de registros

Diagrama de flujo



$A > B$
 $A \geq C$
 $A = .32$
 $< A \quad B \leq A$

Comparación de registros

Ejercicio

Se tiene un registro llamado TEMP y SETPOINT escribir un código con la mínima cantidad de instrucciones para que:

- $TEMP \leq SETPOINT$ Colocar el bit 5 al PORTB a uno en caso contrario colocar a cero
- $IF SETPOINT \geq 22 \text{ AND } SETPOINT \leq 28$
 $THEN PORTB \text{ bit } 5 = 1 \text{ ELSE } PORTB \text{ bit } 0 = 1$
- $IF SETPOINT > TEMP \text{ THEN } TEMP = TEMP + 32 \text{ ELSE REPEAT}$
- $IF (TEMP \leq 10 \text{ AND } 16 \geq SETPOINT) \text{ OR } SETPOINT = 0 \text{ THEN } PORTB \text{ bit } 1 = 0$
 $ELSE PORTB \text{ bit } 1 = 1$

$A > B$
 $A \geq C$
 $A = .32$
 $< A \quad B \leq A$

Comparación de registros

Ejercicio

- Soluciones

```

a) Rutina  MOVF  SETPOINT,W
            SUBWF TEMP,W
            BTFSC STATUS,Z
            GOTO CERO
            BTFSS STATUS,C
            GOTO UNO
            ; GOTO CERO
  
```

```

CERO  BCF  PORTB,5
      GOTO Rutina
  
```

```

UNO   BSF  PORTB,5
      GOTO Rutina
  
```

```

b) Rutina  MOVW  .22
            SUBWF SETPOINT,W
            BTFSS STATUS,C
            GOTO BIT_CERO
            MOVW  .25
            SUBWF SETPOINT,W
            BTFSC STATUS,C
            GOTO BIT_CERO
  
```

```

      BSF  PORTB,5
      GOTO Rutina
  
```

```

BIT_CERO BSF  PORTB,0
  
```

```

c) Rutina  MOVF  TEMP,W
            SUBWF SETPOINT,W
            BTFSC STATUS,Z ; Z=0
            GOTO Rutina
            BTFSS STATUS,C ; C=1
            GOTO Rutina
  
```

```

      MOVW  .32
      ADDWF TEMP,F
      GOTO Rutina
  
```

$A > B$
 $A \geq C$
 $A = .32$
 $< A \quad B \leq A$

Comparación de registros

Ejercicio

```

ROUTINA  MOVW    .10
          SUBWF   TEMP,W
          BTFSC   STATUS, C ; C=0
          GOTO    OTRA
          MOVF    SETPOINT,W
          SUBW    .16
          BTFSS   STATUS, C  ? C=1
          GOTO    OTRA

CERO      BCF     PORTB, I
          GOTO    ROUTINA

OTRA      MOVF    SETPOINT, F
          BTFSC   STATUS, Z  ? Z=1
          GOTO    CERO

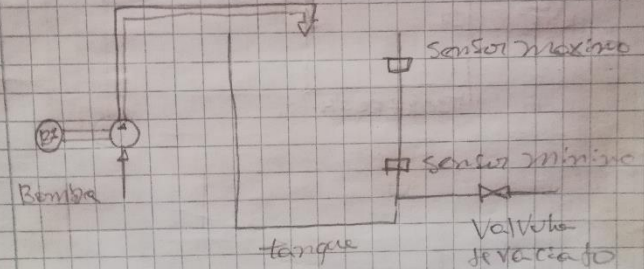
          BSF     PORTB, I
          GOTO    ROUTINA
  
```

$A > B$
 $A \geq C$
 $A = 32$
 $A < B$

Comparación de registros

Ejercicio

- Diseñe un control de nivel para un tanque. Al pulsar Inicio, se activa la bomba BI. Al permanecer encendida hasta alcanzar el nivel máximo, mediante el cual se apagara. Luego se debe abrir la válvula de vaciado. La bomba BI se volverá activar de forma automática cuando se alcance el nivel mínimo procediendo a cerrar la válvula de vaciado, hasta que alcance el nivel máximo, repitiendo el ciclo de forma automática sin necesidad de volver a pulsar Inicio.



Nota: Sensor seco $\rightarrow 0V$
 Sensor mojado $\rightarrow 5V$

Bomba off $\rightarrow 0V$
 Bomba on $\rightarrow 5V$

Válvula cerrada $\rightarrow 0V$
 Válvula abierta $\rightarrow 5V$

- Dibuje el circuito electrónico del MC
- Realice un diagrama de flujo del programa
- Escriba el código del programa en lenguaje C

Comparación de registros

Ejercicio

